Continental
The Future in Motion

QNX security

Florin Anton

www.continental.com/ro-ro

Agenda

1. Introduction
2. QNX Neutrino
3. Embedded security is a hardware-software partnership
    1. Root of trust
    2. Secure boot
    3. Trusted execution environment
    4. Trusted platform module
4. Essential defense mechanisms of a secure OS
    1. Executable space protection
    2. Address space layout randomization
    3. Stack canaries
5. A secure OS can help solve common embedded security challenges

Introduction

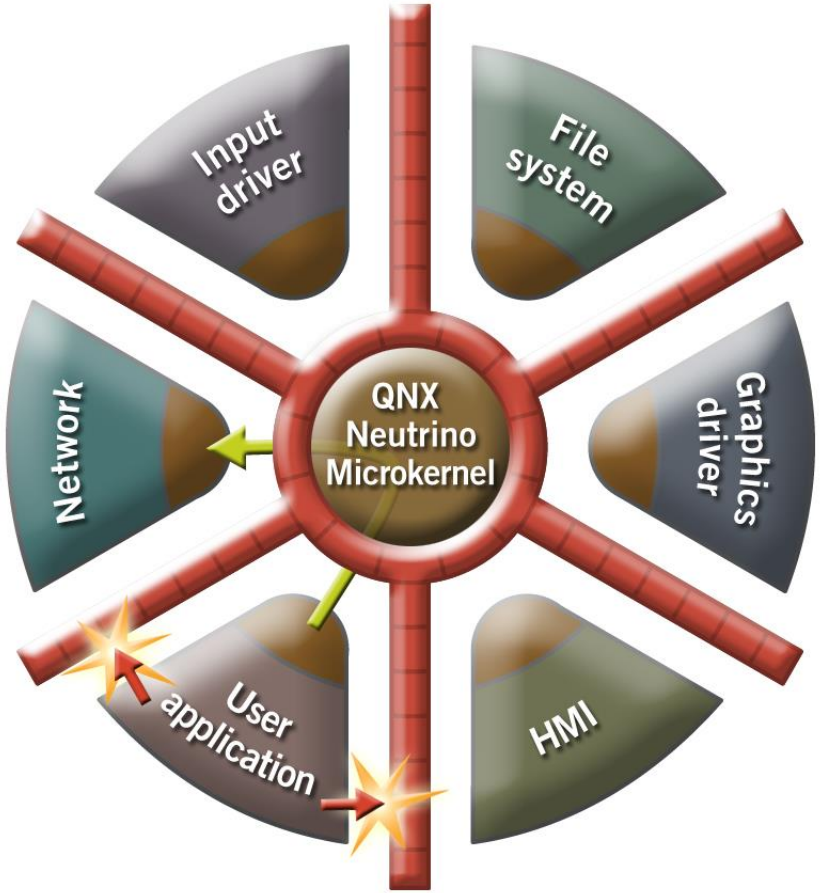
Embedded systems security must be addressed in a holistic manner with best practices throughout the software development life cycle

you'll learn about:

the security advantages of a microkernel OS,
the hardware-software partnership,
essential defense mechanisms
mitigations of common embedded security challenges,

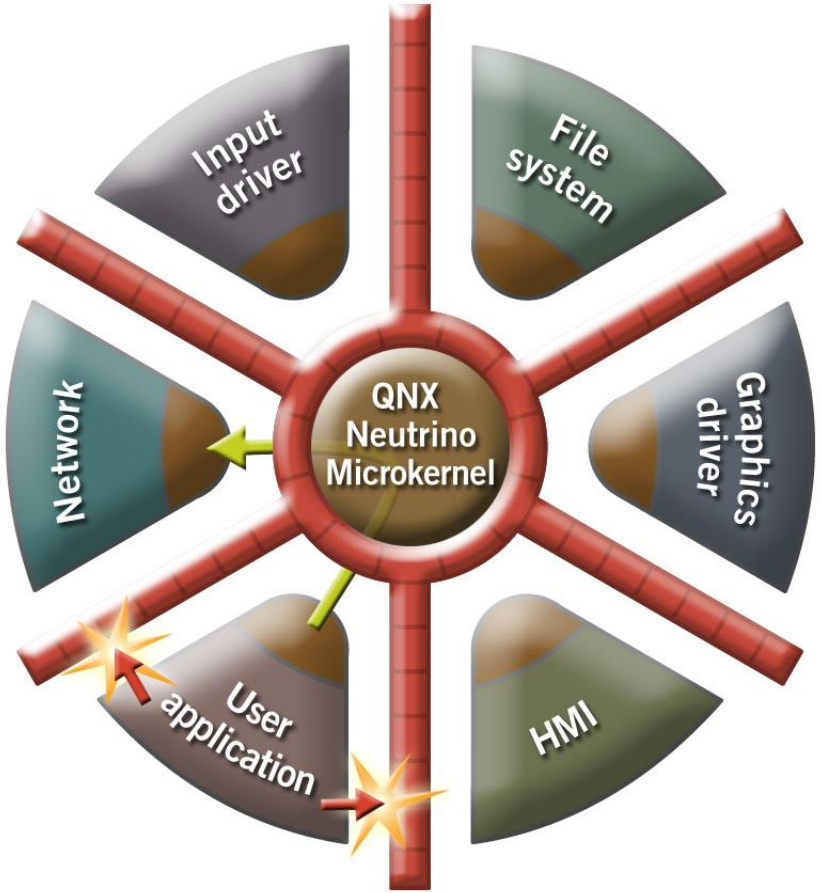
Security advantages of microkernel OS

- A microkernel OS is structured with a tiny kernel space with services like file systems provided in user space, drivers or network stacks.
- Less code running in kernel space reduces the attack surface and increases security.
- The microkernel works with a team of optional cooperating processes that run outside kernel space (in the user space) and provide higher-level OS functionality. Only the core OS kernel is granted access to the entire system, and component isolation prevents an error in one component from affecting other parts of the system. A determined hacker, as long as they don't have root access, can only crash one component at a time when the system runs a microkernel OS or a secure embedded hypervisor.
- The microkernel design and modular architecture of the QNX Neutrino real-time operating system enables BlackBerry QNX customers to create compelling, safe, and secure devices built on a highly reliable OS, a software foundation that helps guard against system malfunctions, malware and security breaches.



Security advantages of microkernel OS

- **Neutrino security in general**
  - Neutrino is a Unix-style operating system, so almost all of the general Unix security information (whether generic, Linux, BSD, etc.) applies to Neutrino as well.
  - Neutrino uses the generic Unix security model of user accounts and file permissions, which is generally sufficient for all our customers. In the embedded space, it's fairly easy to lock down a system to any degree without compromising operation.
  - Neutrino is potentially vulnerable to most of the same threats that other Unix-style systems face. In addition, there are also some issues that are unique to Neutrino.
    - **Message passing**
    - **pdebug**
    - **qconn**
    - **Qnet**
    - **IPSec**



Embedded security is a hardware-software partnership

Root of trust

- A hardware security module (HSM) or hardware root of trust manages keys, performs encryption and decryption functions, and embeds keys for OS and application use. Often these system-on-a-chip (SoC) components provide CPU offload for bulk encryption and decryption, and they may also be used to offload network cryptographic functions.
- Hardware roots of trust are increasingly available as part of the SoC but can also be integrated using discreet electronics, such as an authentication IC or a TPM. During manufacturing, a private key can be generated on a chip or injected into each chip to serve as a root of trust. When the private key is certified by a public key infrastructure (PKI), the secure device identifier can become a foundational component of trusted device connectivity. For example, a secure device identifier can help a developer or designer establish trusted communications with peer devices and cloud-based services.

Secure boot

- Secure boot leverages the signature provided by a device trust anchor, the public part of the root of PKI used to sign device code. When the embedded system boots, the boot image will be validated using this public key and the corresponding trust chain to ensure that boot-time software has not been tampered with. Establishing the provenance of the original software and of any software updates typically relies on digital signatures from a public key cryptosystem. But in some instances, a hybrid model can be used. In a hybrid model, symmetric key cryptography is used to validate software integrity and speed the boot code verification process for time-critical startup requirements. Unlike code verified with a public key, the symmetric key must remain secret, known only to the device.

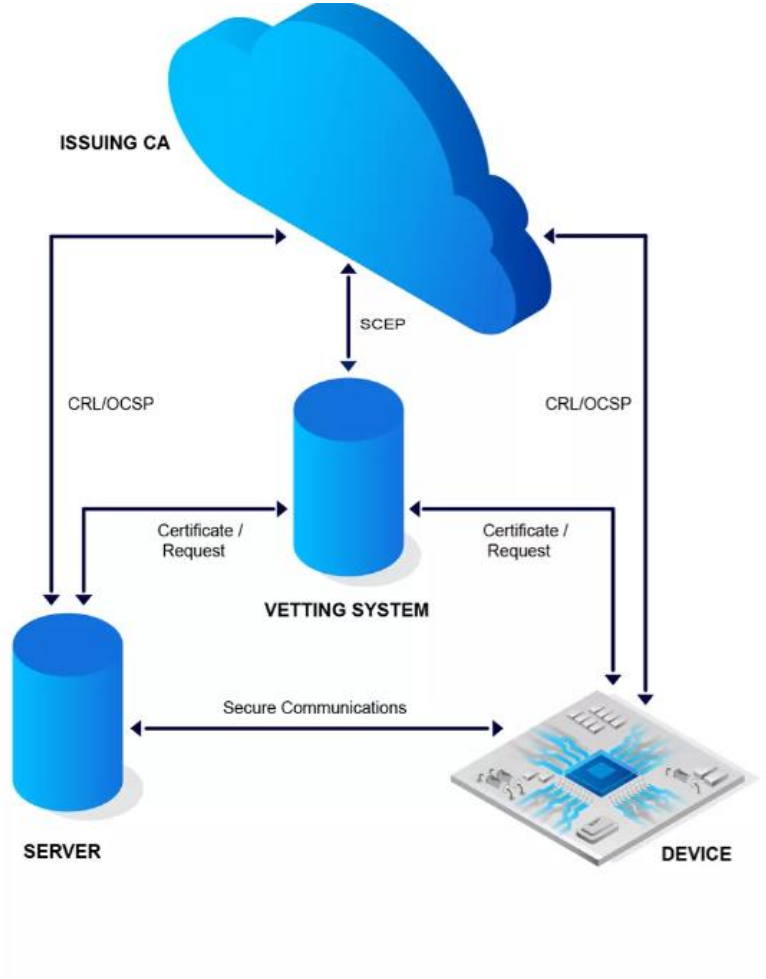
Embedded security is a hardware-software partnership

Trusted execution environment

›A trusted execution environment (TEE) or hardware security zone provides hardware-enforced isolation in a secure area built into the main processor, which allows the software developer to establish a device root of trust. A TEE may run in a secure mode of the processor (e.g., ARM TrustZone) or on a separated, isolated CPU core that acts as a security co-processor to the SoC. TEEs typically allow trusted applications to perform security-critical processing on behalf of the embedded system.

Trusted platform module

›A trusted platform module (TPM) provides hardware-based security functions such as a cryptoprocessor to generate, store and use internal cryptographic keys; encryption of keys and other sensitive material stored in device memory; and measurement and attestation of the integrity of a system state during the boot process.



The process of issuing certificates can be automated with a trusted vetting system verifying all certificate requests.

Essential defense mechanisms of a secure OS

Executable space protection

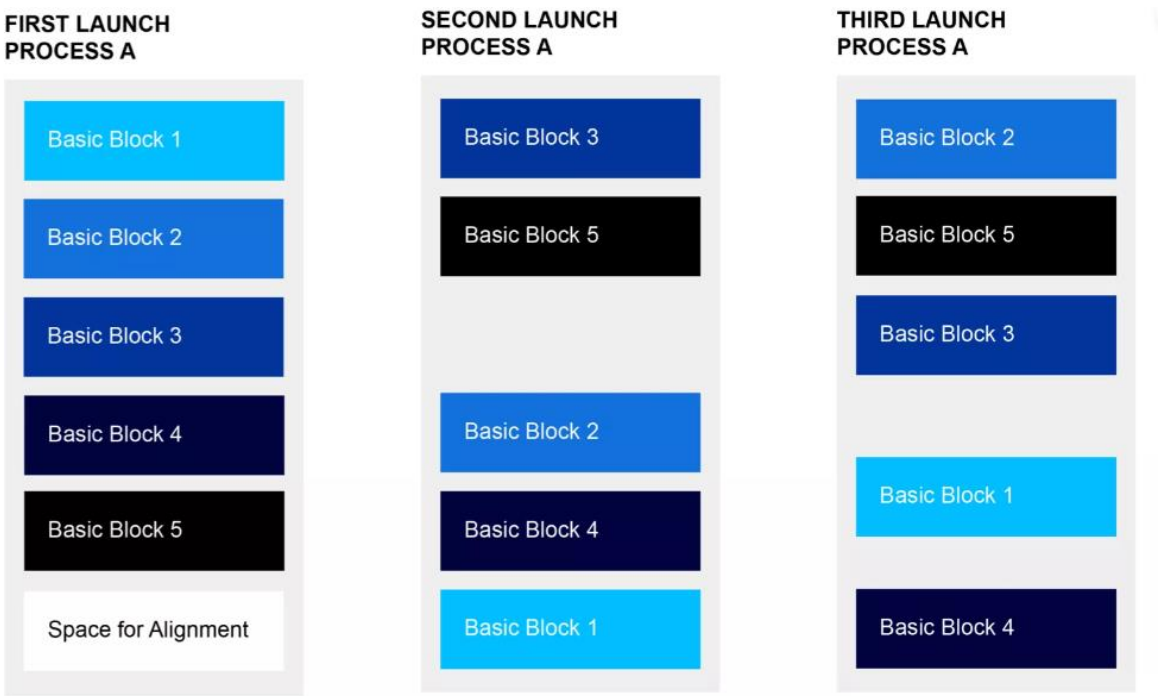
- Executable space protection (ESP) marks specific memory regions as non-executable, so that an attempt to execute machine code in those regions causes an exception.

Address space layout randomization

- Address space layout randomization (ASLR) allocates the base address of the stack, heap and shared memory regions to new locations every time a new process is executed, making buffer overflow attacks difficult because a threat actor can’t predict where the information will be stored.

Stack canaries

- Stack canaries allow the operating system to detect a stack buffer overflow before executing malicious code. The OS places a small random integer before the stack return pointer and checks for it before overwriting memory. If the stack value has changed, the OS will stop execution and cause an exception.



A secure OS can help solve common embedded security challenges

Challenge	Goal	Mitigation
Data Confidentiality at Rest	Prevent a threat actor from seeing, modifying, or exfiltrating sensitive data on the system	File-based encryption
Data Integrity	Package system assets in an integrity-protected container that can be mounted at any time on the system for access	Mark all executables as non-writable Use an integrity-protected disk solution Secure boot
Unrestricted Access to System Resource Managers	Prevent unauthorized system components from accessing system resource manager channels or restrict the operations they can request after they connect	Security policies POSIX permissions and access control lists (ACLs)
Filesystem Object Access Control	Restrict access to filesystem objects by various processes	POSIX permissions and access control lists (ACLs)
Untrusted Code Execution	Prevent a threat actor from running or loading an untrusted binary from a filesystem	Use a utility to mark files and filesystems as trusted
Redirect Control Flow	Prevent a threat actor from modifying executable control flow	Use a compiler to mark the relocation sections of an executable as read-only
Repeatability of Attacks	Make it harder for an attacker to guess where code is loaded in memory	Address space layout randomization (ASLR)

Challenge	Goal	Mitigation
Buffer Overflows	Instrument code to mitigate potential buffer overflow attacks	Use a compiler with fortified function support to detect out-of-bounds memory accesses
Stack Overflows	Instrument code to mitigate stack overflow attacks	Compile code with stack canaries
Revealing Sensitive System Information	Prevent a threat actor from being able to inspect the private information of other processes on the system	Secure the /proc filesystem Security policies
Process Execution with Least Privileges	Configure the system to restrict privileges on processes to make sure that they execute with the least privileges necessary for their tasks	Security policies Use a granular system of policies to govern which operations a particular process is permitted to do POSIX permissions and access control lists (ACLs)
Device Hardware Access to Kernel Memory	Prevent a threat actor from using a device (such as a GPU or network card) to directly memory access (DMA) addresses to which the device has not been explicitly granted access	Use a system memory management unit manager that uses DMA containment
Denial of Service	Prevent a threat actor from interfering with critical systems by exhausting system resources	Set resource limits, such as setrlimit() system calls in C

Thank you!